



LINUX
SECURITY
SUMMIT
NORTH AMERICA

SyzDescribe: Principled, Automated, Static Generation of Syscall Descriptions for Kernel Drivers

Yu Hao¹, Guoren Li¹, Xiaochen Zou¹, Weiteng Chen¹, Shitong Zhu¹, Zhiyun Qian¹, Ardalan Amiri Sani²

¹University of California, Riverside, ²University of California, Irvine

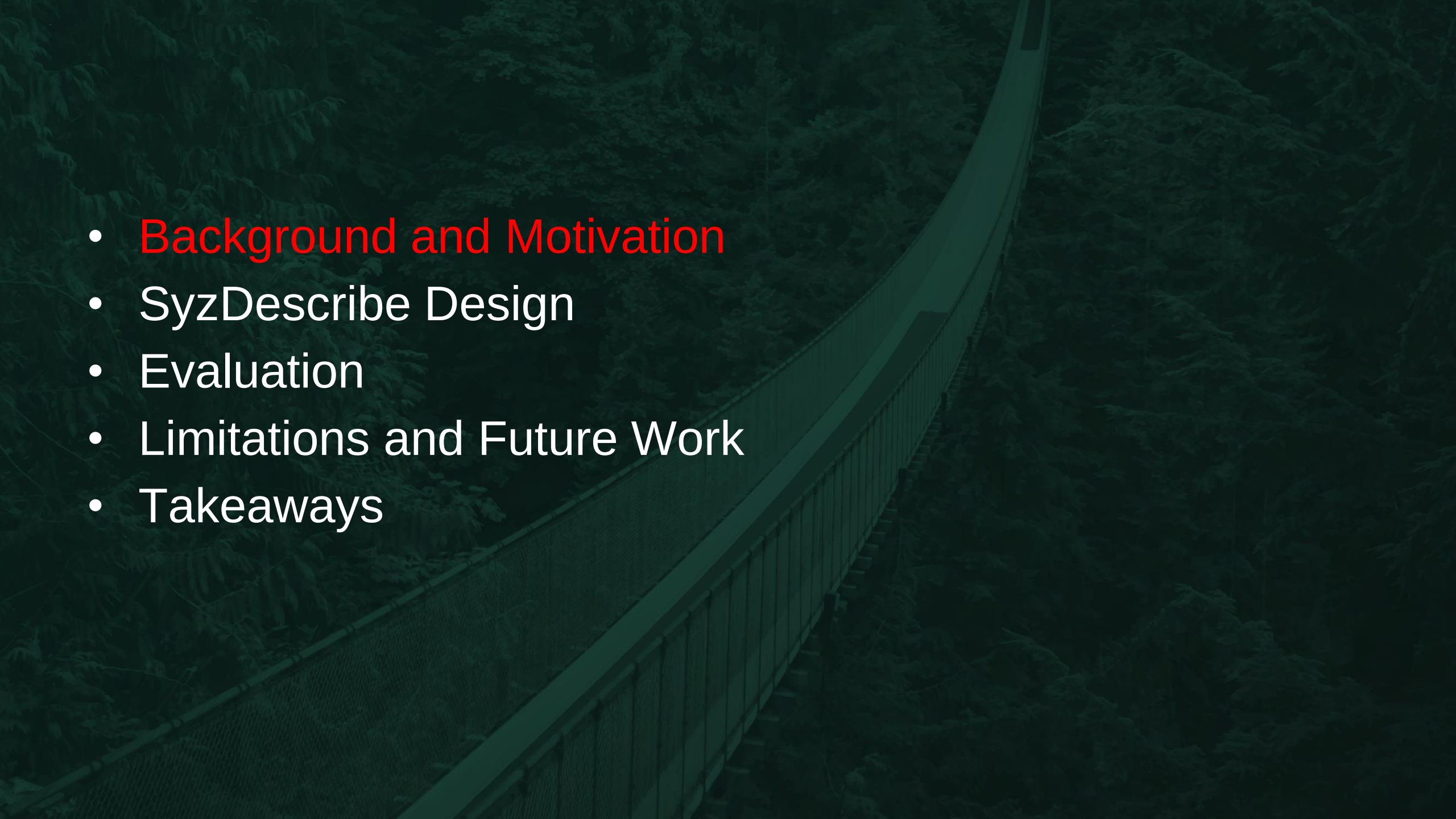
¹{yhao016, gli076, xzou017, wchen130, szhu014}@ucr.edu, zhiyunq@cs.ucr.edu, ²ardalan@uci.edu



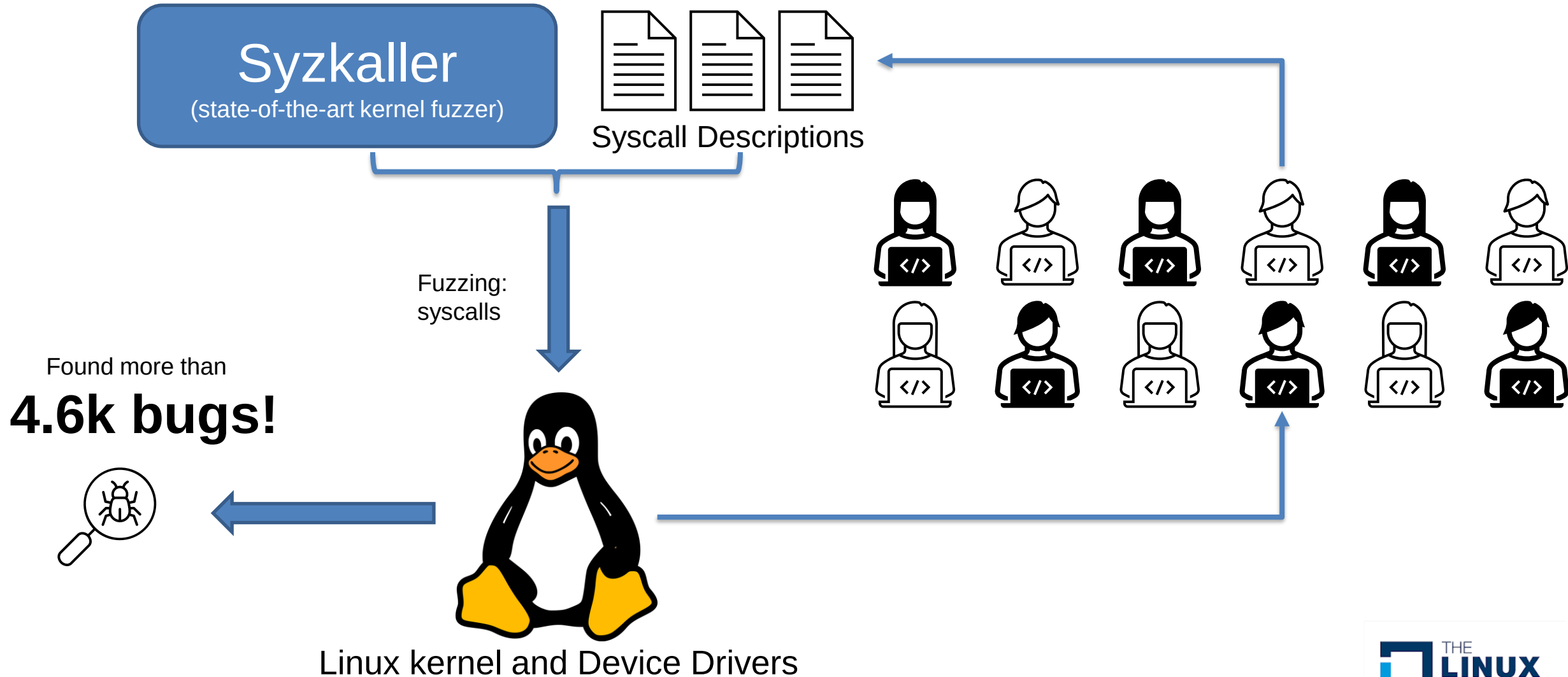
About Yu Hao

- A final year Ph.D. Candidate at UC Riverside under the supervision of Professor Zhiyun Qian
- Focus on Linux kernel security, kernel fuzzing, symbolic execution and static analysis
- Published papers in top academic conferences of computer security (S&P, CCS, NDSS) and software engineering (ICSE, FSE).
- yhao016@ucr.edu



- 
- Background and Motivation
 - SyzDescribe Design
 - Evaluation
 - Limitations and Future Work
 - Takeaways

Kernel Fuzzing, Syzkaller & Syscall Descriptions



Syscall Descriptions

- Syscalls interface
- Device file name
- Command value
- Argument type
- Explicit dependency

non-open file descriptor dependency: fd_kvmvm

```
1. resource fd_kvm[fd]
2. resource fd_kvmvm[fd]
3. open$kvm(fd const[AT_FDCWD], file ptr[in, string["/dev/kvm"]],
flags flags[open_flags],...) fd_kvm
4. ioctl$KVM_CREATE_VM(fd fd_kvm,
cmd const[KVM_CREATE_VM],...) fd_kvmvm
5. ioctl$KVM_SET_USER_MEMORY_REGION(fd fd_kvmvm,
cmd const[ KVM_SET_USER_MEMORY_REGION],
arg ptr[in, kvm_userspace_memory_region])
6. kvm_userspace_memory_region {
7. slot      flags[kvm_mem_slots, int32]
8. flags     flags[kvm_mem_region_flags, int32]
9. paddr     flags[kvm_guest_addrs, int64]
10. size     len[addr, int64]
11. addr     vma64[1:2]
12. }
```

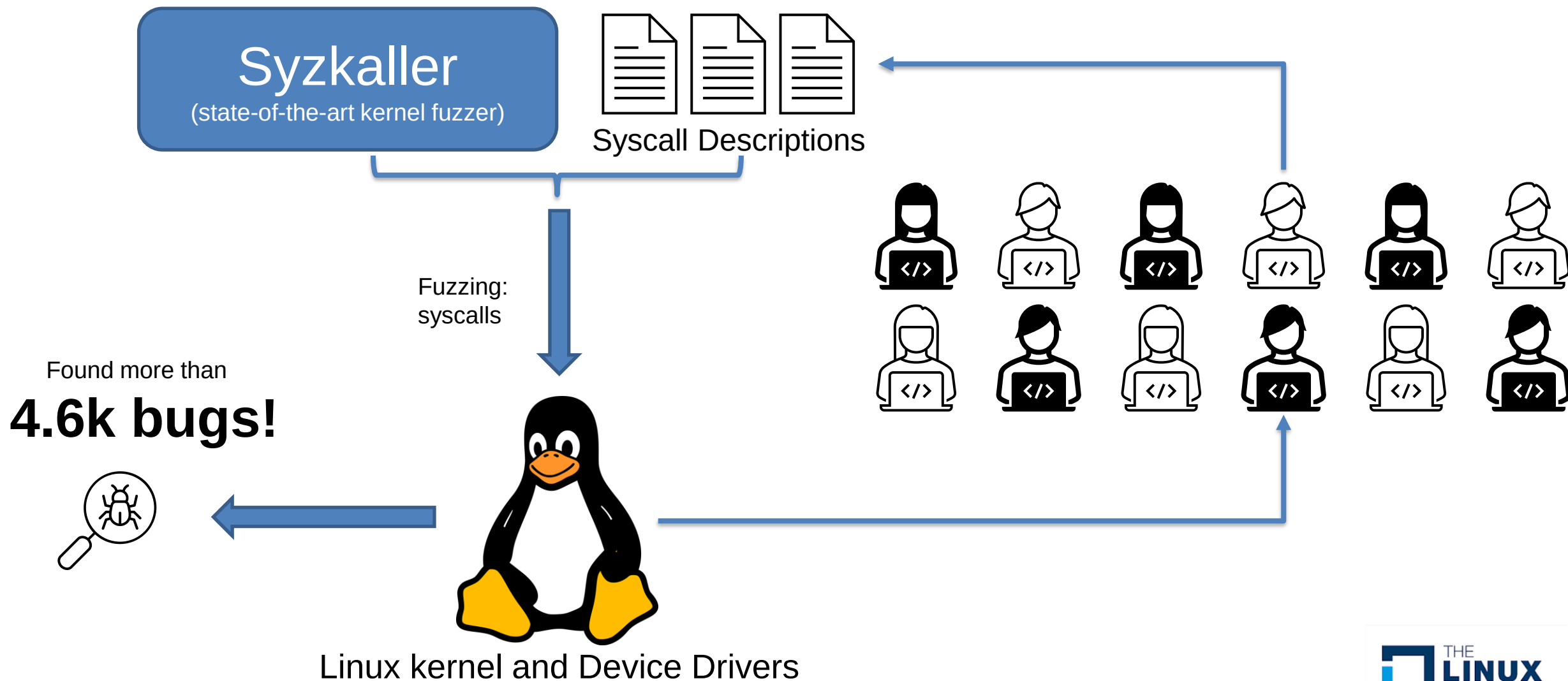
Syscall Descriptions

- Syscalls interface
- Device file name
- Command value
- Argument type
- Explicit dependency

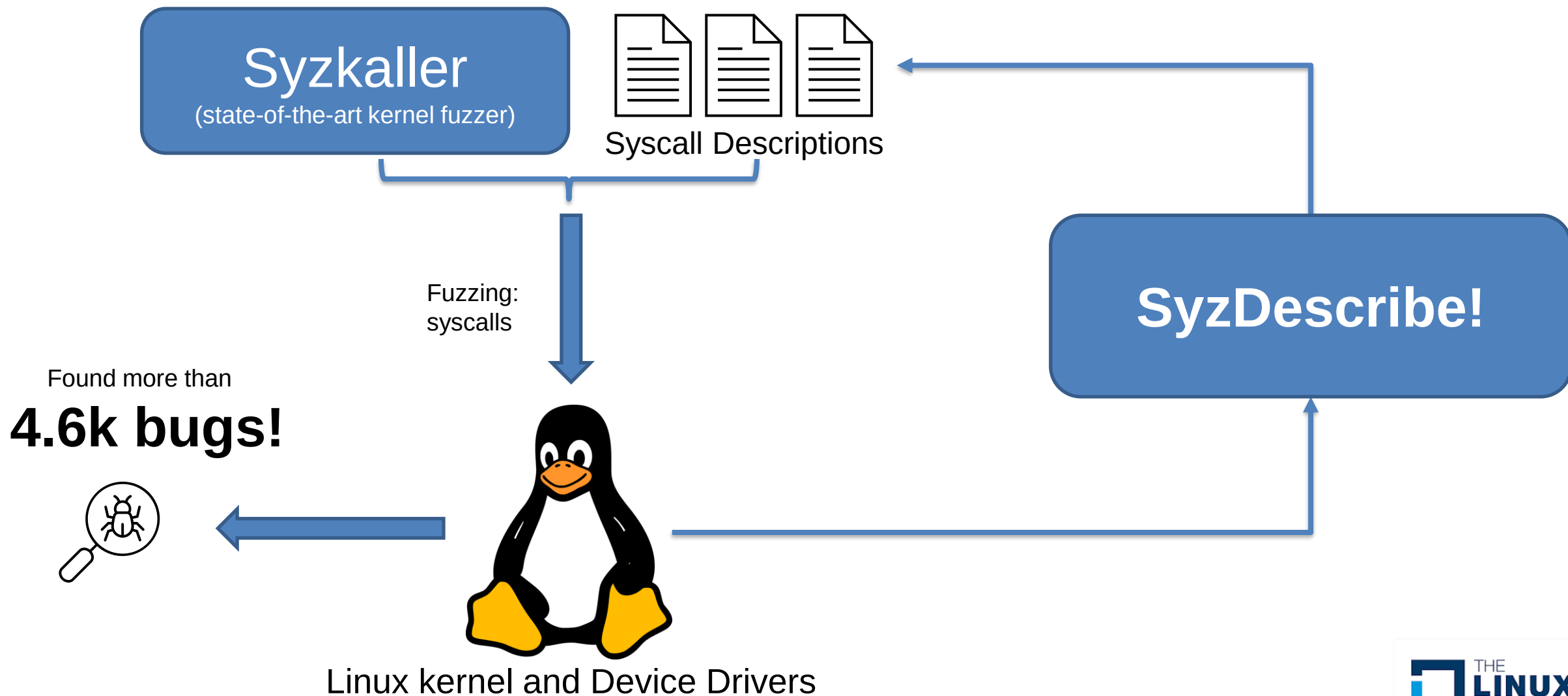
non-open file descriptor dependency: fd_kvmvm

```
1. resource fd_kvm[fd]
2. resource fd_kvmvm[fd]
3. open$kvm(fd const[AT_FDCWD], file ptr[in, string["/dev/kvm"]],
flags flags[open_flags],...) fd_kvm
4. ioctl$KVM_CREATE_VM(fd fd_kvm,
cmd const[KVM_CREATE_VM],...) fd_kvmvm
5. ioctl$KVM_SET_USER_MEMORY_REGION(fd fd_kvmvm,
cmd const[ KVM_SET_USER_MEMORY_REGION],
arg ptr[in, kvm_userspace_memory_region])
6. kvm_userspace_memory_region {
7. slot      flags[kvm_mem_slots, int32]
8. flags     flags[kvm_mem_region_flags, int32]
9. paddr     flags[kvm_guest_addrs, int64]
10. size     len[addr, int64]
11. addr     vma64[1:2]
12. }
```


Our Goal



Our Goal



Key Insight

- At its core, we summarize and model the key invariants and programming conventions regarding kernel driver development.
- This allows us to understand programmatically how a kernel driver is initialized and how its associated interfaces are constructed.
- At a high level, we can then statically reconstruct the initialization of a kernel driver by faithfully following the summarized initialization process

Linux Device Drivers

Results of `ls -al` in “/dev”

Device driver {
Character
Block
Net
(not consider
in this work)

Device number {
Major number
Minor number

Device file name

```
brw-rw---- 1 root disk      7,      5 Apr 18 21:42 loop5
brw-rw---- 1 root disk      7,      6 Apr 18 21:42 loop6
brw-rw---- 1 root disk      7,      7 Apr 18 21:42 loop7
brw-rw---- 1 root disk      7,      8 Apr 24 01:59 loop8
crw-rw---- 1 root disk    10,    237 Apr 25 15:18 loop-control
drwxr-xr-x 2 root root      10,     60 Apr 18 21:42 mapper/
crw----- 1 root root      10,    227 Apr 18 21:42 mcelog
crw-r----- 1 root kmem      1,      1 Apr 18 21:42 mem
drwxrwxrwt 2 root root      40,     40 Apr 18 21:42 mqueue/
drwxr-xr-x 2 root root      60,     60 Apr 18 21:42 net/
crw----- 1 root root   239,      0 Apr 18 21:42 ng0n1
crw-rw-rw- 1 root root      1,      3 Apr 18 21:42 null
crw----- 1 root root   240,      0 Apr 18 21:42 nvme0
brw-rw---- 1 root disk   259,      0 Apr 18 21:42 nvme0n1
brw-rw---- 1 root disk   259,      1 Apr 18 21:42 nvme0n1p1
crw----- 1 root root     10,    144 Apr 18 21:42 nvram
crw-r----- 1 root kmem      1,      4 Apr 18 21:42 port
crw----- 1 root root    108,      0 Apr 18 21:42 ppp
crw----- 1 root root   247,      0 Apr 18 21:42 pps0
crw----- 1 root root     10,      1 Apr 18 21:42 psaux
crw-rw-rw- 1 root tty       5,      2 May  1 2023 ptmx
crw----- 1 root root   246,      0 Apr 18 21:42 ptp0
drwxr-xr-x 2 root root      0,      0 Apr 18 21:42 pts/
crw-rw-rw- 1 root root      1,      8 Apr 18 21:42 random
crw-rw-r-- 1 root root     10,    242 Apr 18 21:42 rfkill
lrwxrwxrwx 1 root root      4,      4 Apr 18 21:42 rtc -> rtc0
crw----- 1 root root   248,      0 Apr 18 21:42 rtc0
```


Linux Device Drivers

- fd = open("device file name")
- ioctl(fd, cmd, arg)

```
if (!filp->f_op->unlocked_ioctl)
    goto out;

error = filp->f_op->unlocked_ioctl(filp, cmd, arg);
```

Define:
module
init
function

Define:
kernel
driver

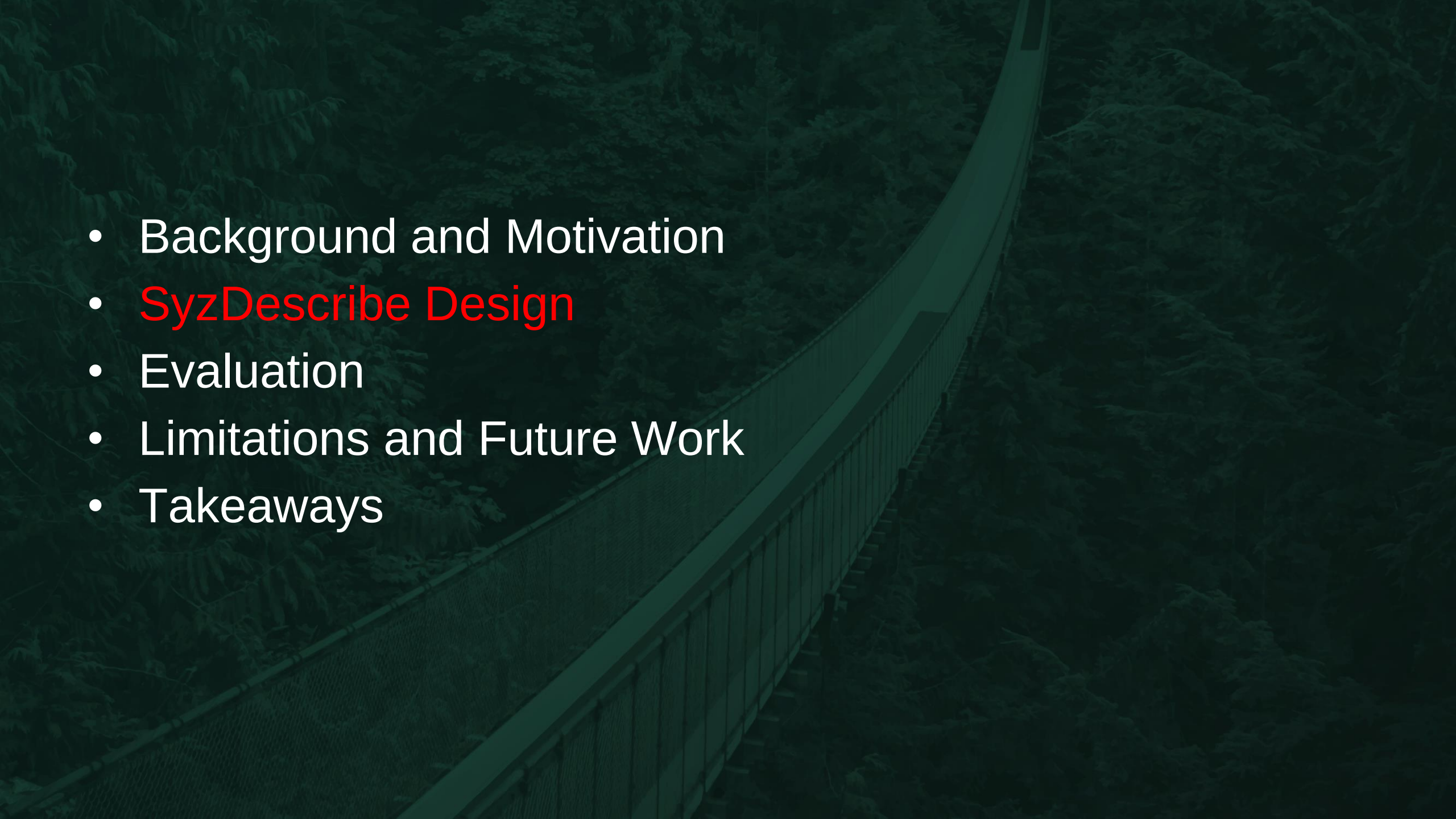
Assign:
function
pointers

Define & Bind:
syscall
handler
structure

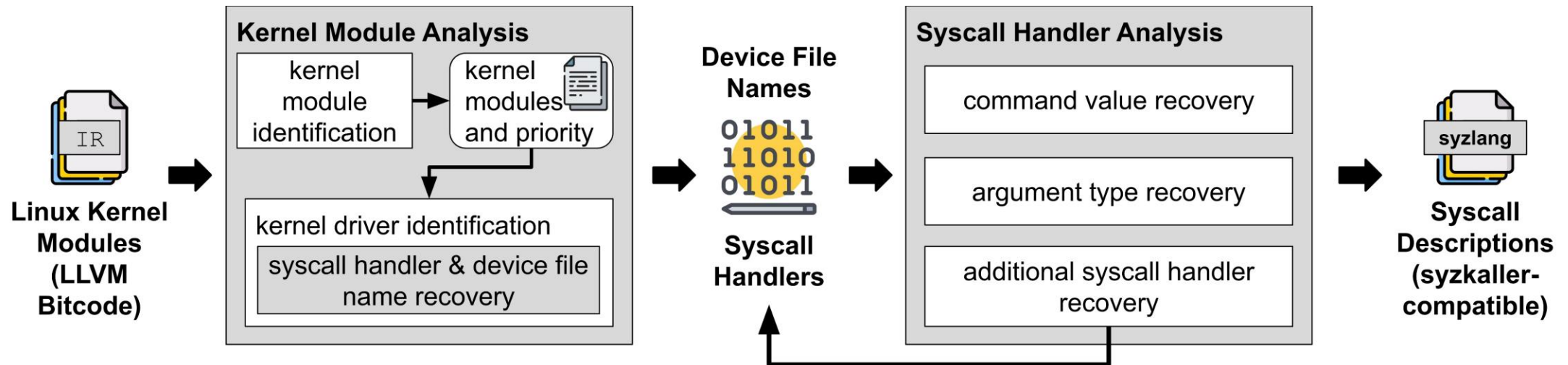
Bind:
device
file name

```
1. static struct xx xx;
2. static struct xx_device_ops xx_ops = {
3.     .ioctl = xx_function_1,
4. }
5. static int init xx init(void) {
6.     dev_t devt = MKDEV(MAJOR, MINOR);
7.     struct cdev *cdev = cdev_alloc();
8.     cdev->dev = devt;
9.     struct device *dev;
10.    device_initialize(dev);
11.    dev->devt = devt;
12.    xx_function_2(cdev);
13.    xx_function_3(dev);
14.    xx.ops = xx_ops;
15. }
16. module_init(xx_init);
17. static struct file_operations ops = {
18.     .open = xx_open,
19.     .unlocked_ioctl = xx_ioctl
20. }
21. void xx_function_2(struct *cdev) {
22.     cdev->ops = ops;
23. }
24. void xx_function_3(struct *dev) {
25.     dev_set_name(dev, "name%d", id);
26. }
27. int xx_open(struct inode *inode, struct
```

Device number

- 
- Background and Motivation
 - **SyzDescribe Design**
 - Evaluation
 - Limitations and Future Work
 - Takeaways

SyzDescribe Design



Kernel Module Analysis

1. Kernel Module Identification.
2. Kernel Driver Identification.

Define:
module
init
function

Define:
kernel
driver

Assign:
function
pointers

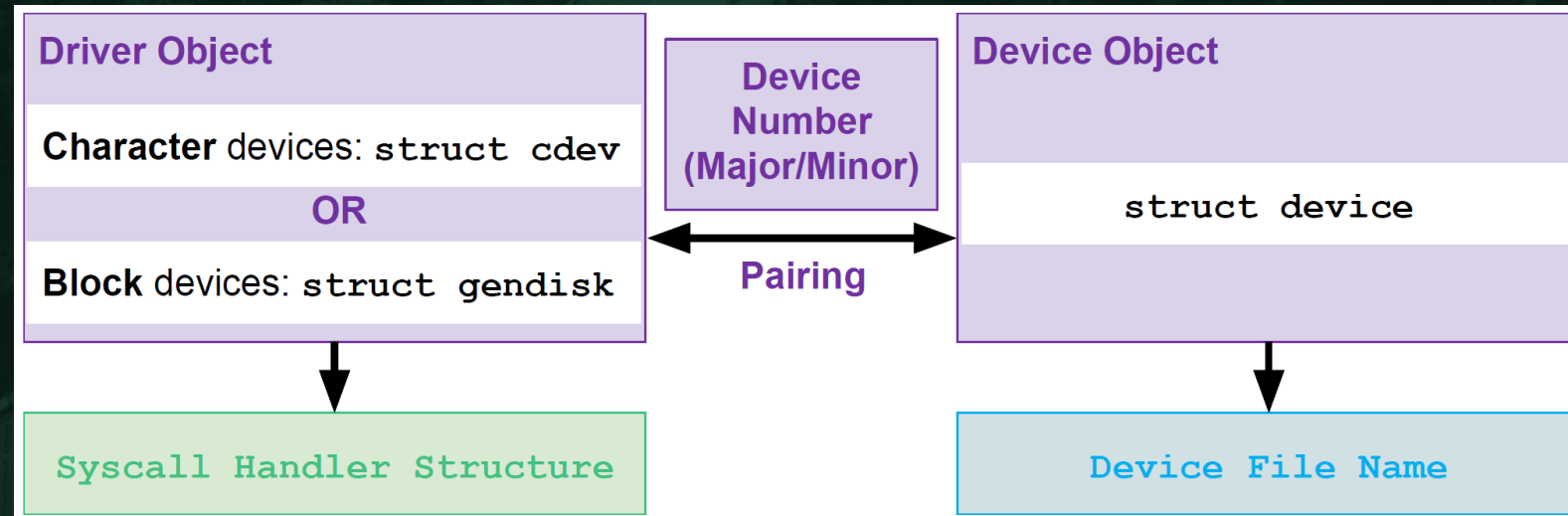
Define & Bind:
syscall
handler
structure

Bind:
device
file name

```
1. static struct xx xx;  
2. static struct xx_device_ops xx_ops = {  
3.     .ioctl = xx_function_1,  
4. }  
5. static int init xx init(void) {  
6.     dev_t devt = MKDEV(MAJOR, MINOR);  
7.     struct cdev *cdev = cdev_alloc();  
8.     cdev->dev = devt;  
9.     struct device *dev;  
10.    device_initialize(dev);  
11.    dev->devt = devt;  
12.    xx_function_2(cdev);  
13.    xx_function_3(dev);  
14.    xx.ops = xx_ops;  
15. }  
16. module_init(xx_init);  
17. static struct file_operations ops = {  
18.     .open = xx_open,  
19.     .unlocked_ioctl = xx_ioctl  
20. }  
21. void xx_function_2(struct *cdev) {  
22.     cdev->ops = ops;  
23. }  
24. void xx_function_3(struct *dev) {  
25.     dev_set_name(dev, "name%d", id);  
26. }  
27. int xx_open(struct inode *inode, struct
```


Kernel Driver Identification.

1. Driver and device object identification and pairing.
2. Syscall handler and device file name recovery.



Kernel Driver Identification.

1. Driver and device object identification and pairing.
2. Syscall handler and device file name recovery.

Define:
module
init
function

Define:
kernel
driver

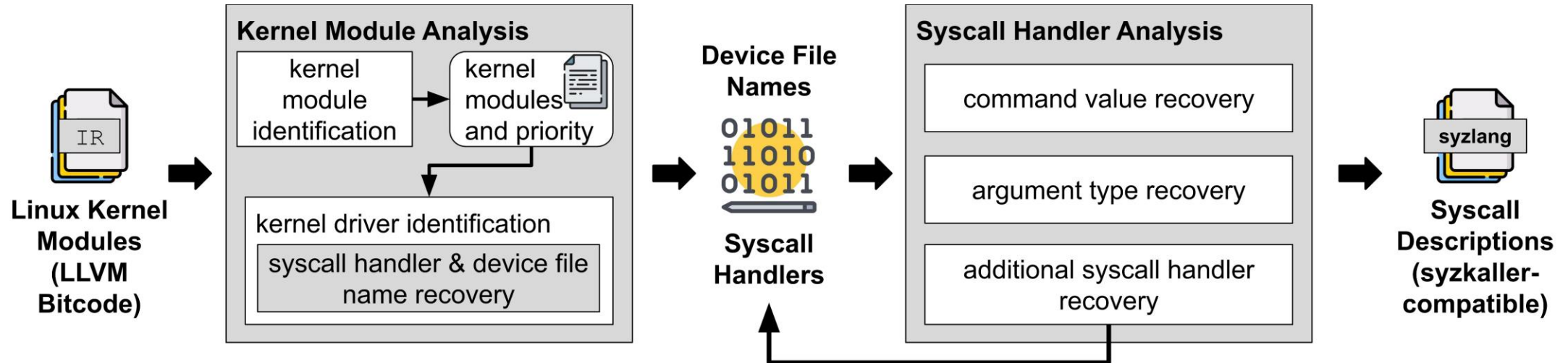
Assign:
function
pointers

Define & Bind:
syscall
handler
structure

Bind:
device
file name

```
1. static struct xx xx;  
2. static struct xx_device_ops xx_ops = {  
3.     .ioctl = xx_function_1,  
4. }  
5. static int init xx init(void) {  
6.     dev_t devt = MKDEV(MAJOR, MINOR);  
7.     struct cdev *cdev = cdev_alloc();  
8.     cdev->dev = devt;  
9.     struct device *dev;  
10.    device_initialize(dev);  
11.    dev->devt = devt;  
12.    xx_function_2(cdev);  
13.    xx_function_3(dev);  
14.    xx.ops = xx_ops;  
15. }  
16. module_init(xx_init);  
17. static struct file_operations ops = {  
18.     .open = xx_open,  
19.     .unlocked_ioctl = xx_ioctl  
20. }  
21. void xx_function_2(struct *cdev) {  
22.     cdev->ops = ops;  
23. }  
24. void xx_function_3(struct *dev) {  
25.     dev_set_name(dev, "name%d", id);  
26. }  
27. int xx_open(struct inode *inode, struct
```

SyzDescribe Design



Syscall Handler Analysis

1. Command Value Recovery.
2. Argument Type Recovery.
3. Additional Syscall Handler Recovery.

Define:
*command
value*

Define:
*argument
type*

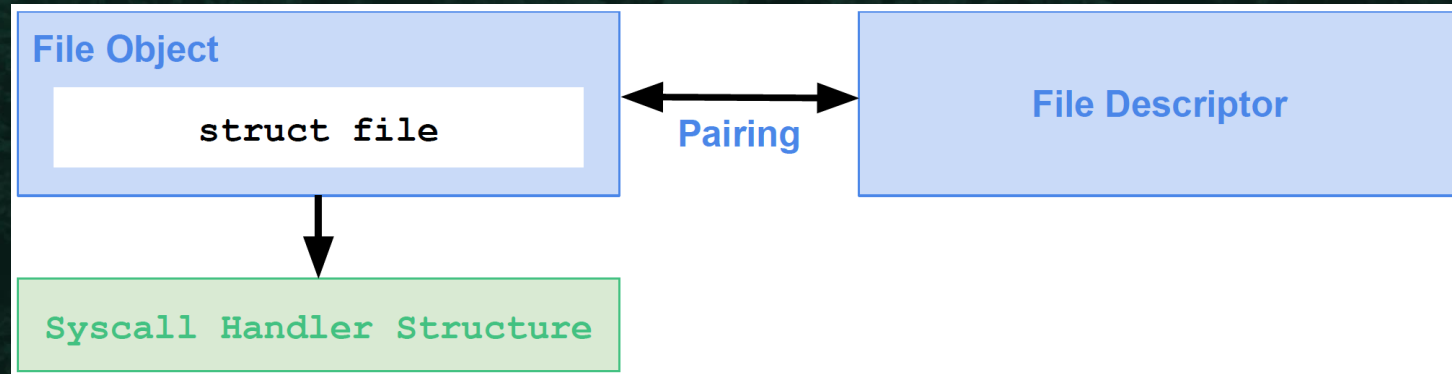
Define:
*additional
syscall
handler*

Use:
*indirect
function
call*

```
27. int xx_open(struct inode *inode, struct
    file *file) {
28.     file->private_data = xx;
29. }
30. long xx_ioctl(struct file *file, int cmd,
    long arg) {
31.     switch (cmd) {
32.         case cmd 1:
33.             struct xx type xx_arg;
34.             copy from user(&xx_arg, arg,
                sizeof(xx_arg));
35.             ...
36.             break;
37.         case cmd 2:
38.             fd = get_unused_fd_flags(...);
39.             file = anon_inode_getfile(...,
                &no_fops, ...);
40.             fd_install(fd, file);
41.             return fd;
42.         default:
43.             xx = file->private_data;
44.             if (xx.ops->ioctl)
45.                 xx.ops->ioctl(file, cmd, arg);
46.     }
47.     if (cmd == cmd 3) {
48.         ...
49.     }
50. }
```

Syscall Handler Analysis

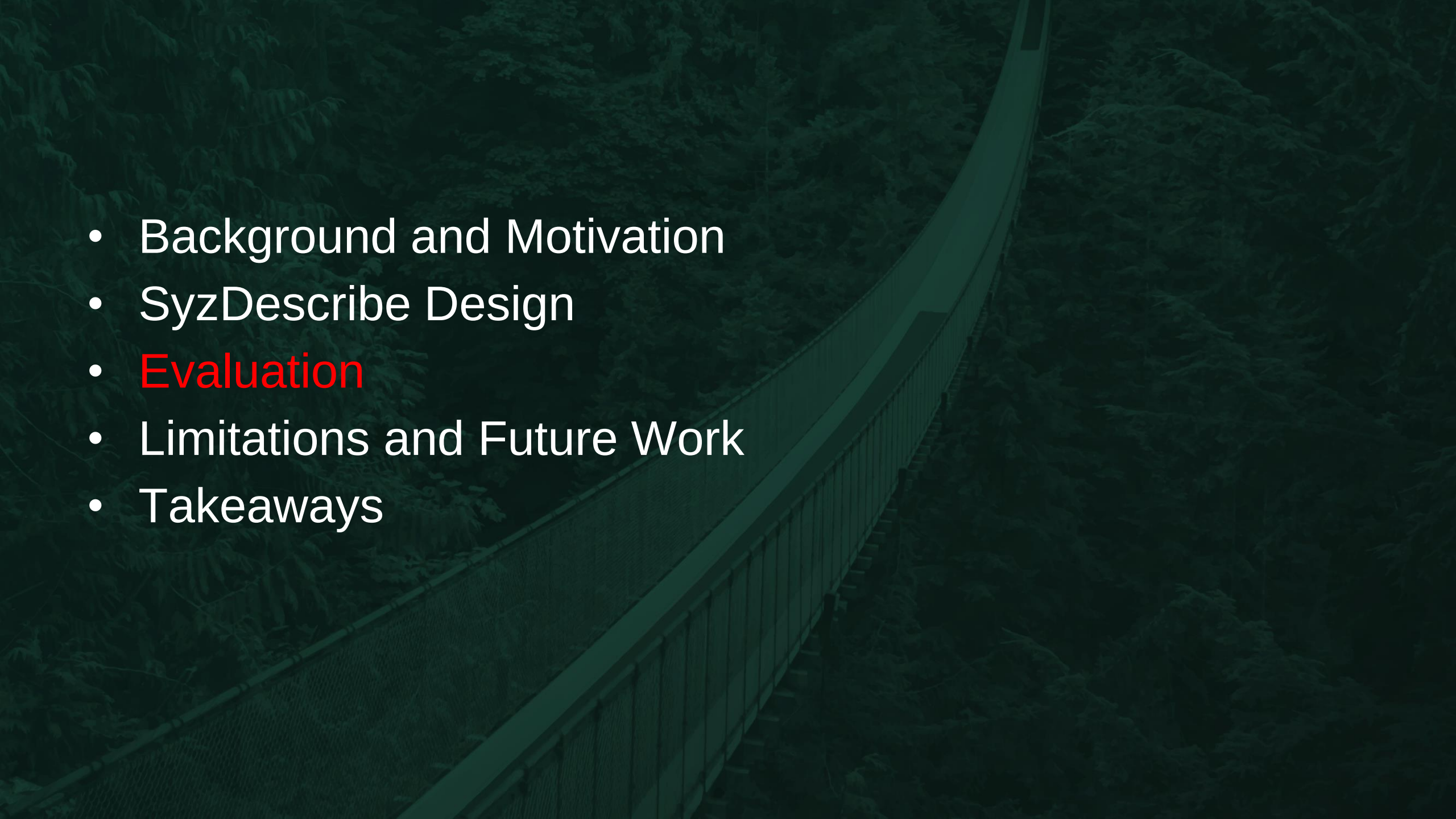
1. Command Value Recovery.
2. Argument Type Recovery.
3. Additional Syscall Handler Recovery.
(non-open file descriptor dependency)



Define:
**additional
syscall
handler**

Use:
*indirect
function
call*

```
37. case cmd_2:
38.     fd = get_unused_fd_flags(...);
39.     file = anon_inode_getfile(...,
40.         &no_fops, ...);
41.     fd_install(fd, file);
42.     return fd;
43. default:
44.     xx = file->private_data;
45.     if (xx.ops->ioctl)
46.         xx.ops->ioctl(file, cmd, arg);
47. if (cmd == cmd_3) {
```


- 
- Background and Motivation
 - SyzDescribe Design
 - **Evaluation**
 - Limitations and Future Work
 - Takeaways

Overall Results

Name	Config	#HANDLER	#NAME	#CMD &TYPE	#N- OPEN	Time (hours)
SyzDescribe	<i>allyes</i>	415	653	3,379	58	3.76
DIFUZE	<i>allyes</i>	4,334	48	560	0	0.42
DIFUZE_F	<i>allyes</i>	4,403	60	761	0	0.16
SyzDescribe	<i>syzbot</i>	185	229	2,119	33	0.47
KSG	<i>syzbot</i>	88	1,098 ¹	714 ²	0	2.38
syzkaller	36c8823	153	232	1,737	27	n/a

- #N-OPEN is the number of non-open file descriptor dependencies
- The *syzbot* config is from syzkaller, which tries to enable as many as possible drivers when fuzzing the Linux kernel through QEMU.
- Analysis time:
 - SyzDescribe spends 3.73 hours to finish the analysis of 7,554 kernel modules under *allyes*, averaging 1.66 seconds per module. The maximum time SyzDescribe needs to spend on one module is 86 seconds.

Current Static Solutions: DIFUZE¹

1. Identify syscall handler structures from a **pre-defined list of struct types**
2. Identify the corresponding device file name (**any constant string**) used **near the reference of the structure**
3. Apply inter-procedural static analysis to syscall handlers to recover the **argument**

```
{ "struct.vfio_iommu_driver_ops", 5, WRITE_HDR},  
{ "struct.vfio_iommu_driver_ops", 6, IOCTL_HDR},  
{ "struct.rtc_class_ops", 0, IOCTL_HDR},  
{ "struct.net_device_ops", 11, IOCTL_HDR},  
{ "struct.kvm_device_ops", 8, IOCTL_HDR},  
{ "struct.ide_disk_ops", 8, IOCTL_HDR},  
{ "struct.ide_ioctl_devset", 0, IOCTL_HDR},  
{ "struct.ide_ioctl_devset", 1, IOCTL_HDR},  
{ "struct.pci_ops", 3, READ_HDR},  
{ "struct.pci_ops", 4, WRITE_HDR},
```

```
static const struct snd_hwdep_ops ops = {  
    .read      = hwdep_read,  
    .write     = hwdep_write,  
    .release   = hwdep_release,  
    .poll      = hwdep_poll,  
    .ioctl     = hwdep_ioctl,  
    .ioctl_compat = hwdep_compat_ioctl,  
};  
struct snd_hwdep *hwdep;  
int err;  
  
err = snd_hwdep_new(card: efw->card, id: "Fireworks", device: 0, rhwdep: &hwdep);  
if (err < 0)
```

¹ "DIFUZE: interface aware fuzzing for kernel drivers," CCS 17

SyzDescribe vs. DIFUZE

	DIFUZE	SyzDescribe
Modeling of kernel drivers?	Pre-defined list of struct types	The well-established contract between the core kernel and drivers.
Recover syscall handlers and device file names reasonably?	X	√
Handle indirect call when recovering the argument?	X	√
Compatible with the newer kernel?	X	√
Recover non-open file descriptor dependencies?	X	√
Directly generate syzkaller compatible syscall descriptions?	X	√

SyzDescribe vs. DIFUZE_F

Name	Config	#HANDLER	#NAME	#CMD &TYPE	#N- OPEN	Time (hours)
SyzDescribe	<i>allyes</i>	415	653	3,379	58	3.76
DIFUZE	<i>allyes</i>	4,334	48	560	0	0.42
DIFUZE_F	<i>allyes</i>	4,403	60	761	0	0.16
SyzDescribe	<i>syzbot</i>	185	229	2,119	33	0.47
KSG	<i>syzbot</i>	88	1,098 ¹	714 ²	0	2.38
syzkaller	36c8823	153	232	1,737	27	n/a

most syscall handler structures found by DIFUZE (i.e., 4334) or DIFUZE_F (i.e., 4403) are actually false positive, given that they recovered only 48 or 60 device file names.

DIFUZE_F: We fixed issues that fall under two categories: (1) correcting hard-coded offsets for certain types that have changed in kernel v5.12; (2) fixing failed tracking of `copy_from_user()` destination object due to changes in compiled LLVM bitcode (otherwise no argument type can be recovered). The fixed version is labeled as DIFUZE_F.

SyzDescribe vs. DIFUZE_F: Accuracy

Name	#HANDLER			#NAME			#CMD			#TYPE			#N-OPEN		
	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁
SyzDescribe	96	0	0.95	74	31	0.71	1,039	48	0.84	521	2	0.74	6	0	1.00
DIFUZE	49	30	0.53	16	4	0.26	269	24	0.32	0	0	0.00	0	0	0.00
DIFUZE_F	52	25	0.57	16	4	0.26	269	26	0.32	78	4	0.16	0	0	0.00
Ground truth	106	-	-	103	-	-	1,400	-	-	894	-	-	6	-	-

- Randomly pick 100 kernel drivers as dataset for comparison against DIFUZE
- For the ground truth collection, to mitigate potential errors we make during the process, we always cross-validate with the syscall descriptions generated with SyzDescribe and from syzkaller (the manually-curated ones) to see whether we missed anything.

SyzDescribe vs. DIFUZE_F: Fuzzing

Device Name	SyzDescribe		DIFUZE_F		Device Name	SyzDescribe		DIFUZE_F	
	#Cov	crash	#Cov	crash		#Cov	crash	#Cov	crash
“loop%d”	18,644	5.3	0	0.0	“uinput”	6,070	0.0	6,136	0.0
“loop-control”	7,799	1.0	7,789	1.0	“ppp”	7,557	0.3	0	0.0
“rtc%d”	14,513	4.0	0	0.0	“ashmem”	3,799	0.0	0	0.0
“sg%d”	17,017	5.3	0	0.0	“fuse”	3,423	0.0	3,603	0.0
“sr%d”	15,554	2.0	0	0.0	“kvm”	16,932	4.0	6,093	1.7
“ptmx”...	15,195	4.0	0	0.0	“btrfs-control”	4,053	0.0	3,684	0.0
“usbmon%d”	13,898	3.7	0	0.0	“capi20”	3,756	0.0	0	0.0
“snapshot”	4,099	0.3	4,070	0.0	“fd%d”	13,872	3.3	0	0.0
“rfkill”	3,427	0.0	3,595	0.0	“mISDNtimer”	3,546	0.0	3,662	0.0
“controlC%d”	14,429	3.3	0	0.0	“vhost-net”	4,469	0.0	4,367	0.0
“timer”	4,364	0.0	0	0.0	“vhost-vsock”	4,398	0.7	4,524	0.0
“nbd%d”	15,606	3.7	0	0.0	“vmci”	6,860	2.0	5,320	1.3
“qat_adf_ctl”	3,779	0.3	0	0.0	“vsock”	3,620	0.0	3,359	0.0
“udmabuf”	2,505	1.0	2,285	0.0	“nvram”	3,732	1.0	3,701	0.0
“i2c-%d”	7,347	1.0	0	0.0	“hpet”	3,254	0.3	3,587	0.0
Sum						247,516	46.7	65,777	4.0

Current Dynamic Solutions: KSG¹

1. Scan all the device files available under `/dev`
2. Hook syscall `open("device file name")` and get `syscall handler structures`
3. Apply `intra-procedural` symbolic execution to syscall handlers to recover the `argument`

```
root disk      7,      5 Apr 18 21:42 loop5
root disk      7,      6 Apr 18 21:42 loop6
root disk      7,      7 Apr 18 21:42 loop7
root disk      7,      8 Apr 24 01:59 loop8
root disk     10,    237 Apr 25 15:18 loop-control
```

```
static const struct file_operations loop_ctl_fops = {
    .open          = nonseekable_open,
    .unlocked_ioctl = loop_control_ioctl,
    .compat_ioctl  = loop_control_ioctl,
    .owner         = THIS_MODULE,
    .llseek        = noop_llseek,
};
```

¹ “{KSG}: Augmenting kernel fuzzing with system call specification generation,” (USENIX ATC 22),

SyzDescribe vs. KSG

	KSG	SyzDescribe
Requirements	Compliable and bootable kernel, Dynamic instrumentation. Not work for single module	Compliable kernel or module
Modeling of kernel drivers?	Limited modeling	The well-established contract between the core kernel and drivers.
Support all complied modules?	X	√
Compatible with kernel with fixed configuration, e.g., Android kernel?	X	√
Handle function call when recovering the argument?	X	√
Recover non-open file descriptor dependencies?	X	√

SyzDescribe vs. KSG

Name	Config	#HANDLER	#NAME	#CMD &TYPE	#N- OPEN	Time (hours)
SyzDescribe	<i>allyes</i>	415	653	3,379	58	3.76
DIFUZE	<i>allyes</i>	4,334	48	560	0	0.42
DIFUZE_F	<i>allyes</i>	4,403	60	761	0	0.16
SyzDescribe	<i>syzbot</i>	185	229	2,119	33	0.47
KSG	<i>syzbot</i>	88	1,098 ¹	714 ²	0	2.38
syzkaller	36c8823	153	232	1,737	27	n/a

#HANDKER: 185 vs. 88

- 1) 64 of 185 handlers which are compiled into the kernel but need to satisfy some other dependencies (e.g., plugin related hardware or execute other syscalls) to make the related device file exposed under the /dev directory.
- 2) 49 of 185 handlers which need additional modeling.
- 3) 33 of 185 handlers are non-open file descriptor dependencies.

¹KSG generates many similar device file names (i.e., “usbmon0”, “usbmon1”, ..., “usbmon40”), which is “usbmon#” in SyzDescribe and syzkaller descriptions. If we group them, the number is 142.

²KSG generates some totally repeated CMD&TYPE. If we remove those repeated ones, the number is 485.

SyzDescribe vs. KSG: Accuracy

Name	#HANDLER			#NAME			#CMD			#TYPE			#N-OPEN		
	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁
SyzDescribe	74	0	0.99	58	24	0.75	837	42	0.81	399	2	0.70	6	0	1.00
KSG	43	2	0.71	45	0	0.77	223	303	0.26	64	15	0.16	0	0	0.00
Ground truth	76	-	-	72	-	-	1,192	-	-	732	-	-	6	-	-

- To be fair against KSG, we choose to include a subset of the 100 drivers that are compiled with the syzbot config, i.e., 70 drivers; the other 30 are compiled only under the allyes config.

SyzDescribe vs. KSG: Fuzzing

- The syscall descriptions generated by KSG in many cases incorrectly associate distinct drivers with the same set of syscall handlers.
- For example, /dev/disks and /dev/dri are two different drivers and should have their own ioctl() handlers and descriptions. However, KSG associate them with the same descriptions (e.g., handlers and command values).
- Such mix-ups make it hard to tease out the descriptions that belong to specific drivers.
- We compare specifically against KSG by fuzzing the whole kernel using all available descriptions

	#Cov	crash
KSG	30,345	7.7
SyzDescribe	34,097	9.0
KSG+syzkaller	50,049	11.3
SyzDescribe +syzkaller	58,201	15.3

SyzDescribe vs. Syzkaller Description (Manual)

Name	#HANDLER			#NAME			#CMD			#TYPE			#N-OPEN		
	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁
SyzDescribe	47	0	0.99	42	7	0.87	807	34	0.81	393	2	0.68	5	0	1.00
syzkaller	45	0	0.97	46	0	0.98	922	0	0.89	506	3	0.80	3	0	0.75
Ground truth	48	-	-	48	-	-	1,141	-	-	755	-	-	5	-	-

- 57 drivers are missed completely in the syzkaller descriptions. to be fair against them, we choose to include a subset of drivers that are covered by them, i.e., 43 drivers.
- Regarding #CMDs, SyzDescribe generated more CMDs for 11 drivers, the same number for 17 drivers, and less for 13 drivers. Regarding #TYPES, SyzDescribe generated more TYPES for 6 drivers, the same number for 24 drivers, and less for 11 drivers. Combined together, SyzDescribe generated either more CMDs or more TYPES for 13 drivers.

“Bugs” in Human-Generated Descriptions

Category	Syscall handler structure	#	Commit time of related code in Linux kernel	Update time of latest syzlang (before 04/2021)
kernel drivers with CMD FN	lo_fops	1	05/2020	12/2019
	sg_fops	7	10/2014	01/2019
	usbdev_file_operations	11	08/2019	01/2020
	rftkill_fops	1	06/2009	03/2019
	snd_timer_f_ops	6	04/2018	03/2020
	snd_ctl_f_ops	1	05/2005	01/2020
	nbd_fops	1	04/2005	02/2021
	raw_fops	19	01/2020	06/2020
	ashmem_fops	2	12/2011	01/2018
	ppp_device_fops	4	12/2020	01/2019
	tun_fops	1	02/2018	03/2020
kernel drivers with TYPE FN	lo_fops	1	05/2020	12/2019
	usbdev_file_operations	5	01/2015	01/2020
	raw_fops	1	10/2015	06/2020
	sr_bdops	9	04/2005	08/2020
	hiddev_fops	5	03/2008	04/2020
	evdev_fops	3	08/2010	03/2020
kernel drivers with TYPE FP	snd_timer_f_ops	2	04/2018	03/2020
	snd_ctl_f_ops	1	12/2019	01/2020
kernel drivers with N-OPEN FN	udmabuf_fops	1	09/2018	02/2019 (fixed in 01/2022)
	lo_fops	1	05/2007	12/2019

- These syscall description “bugs” can persist over an extended period of time.
- We have reported all the bugs to syzkaller (all of which are fixed) and shared the syscall descriptions generated by SyzDescribe.

SyzDescribe vs. Ground Truth (Final Goal) : Accuracy

Name	#HANDLER			#NAME			#CMD			#TYPE			#N-OPEN		
	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁	TP	FP	F ₁
SyzDescribe	96	0	0.95	74	31	0.71	1,039	48	0.84	521	2	0.74	6	0	1.00
Ground truth	106	-	-	103	-	-	1,400	-	-	894	-	-	6	-	-

- Syscall handler: The major reason for the false negatives (9 out of 10) is that syscall handler structures take dynamically constructed variables (as opposed to target functions directly),
- Device file name:
 - The main reason for the false negatives of #NAME (28 out of 29 false negatives) is that the device file names are constructed dynamically (not following the standard functions with format strings that we model).
 - As for the false positives of #NAME, the reason is that their major number/device number are generated dynamically, so we can not match the driver object and device object accurately.

SyzDescribe vs. Ground Truth (Final Goal) : Accuracy

- Command value:
 - The false negatives of #HANDLER
 - Non-trivial (custom) uses of the command values
- Argument type:
 - The false negatives of #HANDLER or #CMD
 - Incomplete data flow tracking
 - The failure to model additional functions (defined in inlined assembly) that copy data from user space, e.g., get_user().

```
if (_IOC_NR(cmd) == _IOC_NR(HIDIOCSFEATURE(0))) {  
    int len = _IOC_SIZE(cmd);  
    ret = hidraw_send_report(file, buffer: user_arg,  
        break;  
}  
    Alan Ott, 12 years ago • HID: Add Support for  
if (_IOC_NR(cmd) == _IOC_NR(HIDIOCGFEATURE(0))) {  
    int len = _IOC_SIZE(cmd);  
    ret = hidraw_get_report(file, buffer: user_arg, c  
        break;  
}
```

```
static const struct ioctl_handler {  
    unsigned int cmd;  
    int (*func)(struct snd_seq_client *client, void *arg);  
} ioctl_handlers[] = {  
    { SNDRV_SEQ_IOCTL_PVERSION, snd_seq_ioctl_pversion },  
    { SNDRV_SEQ_IOCTL_CLIENT_ID, snd_seq_ioctl_client_id },  
    { SNDRV_SEQ_IOCTL_SYSTEM_INFO, snd_seq_ioctl_system_info },  
    { SNDRV_SEQ_IOCTL_RUNNING_MODE, snd_seq_ioctl_running_mode },  
    { SNDRV_SEQ_IOCTL_GET_CLIENT_INFO, snd_seq_ioctl_get_client_info },
```


SyzDescribe vs. Syzkaller Description vs. Ground Truth: Fuzzing

Device Name	SyzDescribe		syzkaller		Ground truth	
	#Cov	crash	#Cov	crash	#Cov	crash
“loop%d”	18,644	5.3	15,016	5.0	18,438	6.3
“loop-control”	7,799	1.0	6,422	0.7	7,800	1.0
“rtc%d”	14,513	4.0	13,061	4.3	14,153	3.0
“sg%d”	17,017	5.3	17,136	6.0	17,307	5.7
“sr%d”	15,554	2.0	15,264	2.0	15,400	2.3
“ptmx”...	15,195	4.0	15,239	5.7	15,833	6.0
“usbmon%d”	13,898	3.7	13,619	1.7	13,717	3.0
“snapshot”	4,099	0.3	3,422	0.0	3,968	0.0
“rfkill”	3,427	0.0	2,276	0.0	3,141	0.3
“controlC%d”	14,429	3.3	13,888	3.3	14,610	3.7
“timer”	4,364	0.0	2,977	0.7	4,334	0.5
“nbd%d”	15,606	3.7	15,423	5.3	15,234	2.3
“qat_adf_ctl”	3,779	0.3	2,545	0.0	4,056	1.0
“udmabuf”	2,505	1.0	1,391	0.0	2,520	1.0
“i2c-%d”	7,347	1.0	12,576	3.7	12,576*	3.7*

Device Name	SyzDescribe		syzkaller		Ground truth	
	#Cov	crash	#Cov	crash	#Cov	crash
“uinput”	6,070	0.0	6,318	1.0	6,003	1.3
“ppp”	7,557	0.3	6,350	0.0	7,605	0.3
“ashmem”	3,799	0.0	3,300	0.0	3,684	0.7
“fuse”	3,423	0.0	1,737	0.0	3,409	0.0
“kvm”	16,932	4.0	21,593	9.7	24,289	7.0
“btrfs-control”	4,053	0.0	0	0.0	4,053*	0.0*
“capi20”	3,756	0.0	0	0.0	3,756*	0.0*
“fd%d”	13,872	3.3	0	0.0	14,127	6.7
“mISDNtimer”	3,546	0.0	0	0.0	3,708	0.0
“vhost-net”	4,469	0.0	0	0.0	4,469*	0.0*
“vhost-vsock”	4,398	0.7	0	0.0	4,398*	0.7*
“vmci”	6,860	2.0	0	0.0	6,154	2.0
“vsock”	3,620	0.0	0	0.0	3,620*	0.0*
“nvram”	3,732	1.0	0	0.0	3,732*	1.0*
“hpet”	3,254	0.3	0	0.0	3,254*	0.3*
Sum	247,516	46.7	189,553	49.0	259,334	59.8

Fuzzing Android Kernel of Pixel 6

- SyzDescribe recovers 154 syscall handlers corresponding to 139 kernel drivers.
- Crashes are captured by RAMDUMP MODE in Pixel kernel (no public documentation about it)
- Difficult to understand the root causes of the bugs.

Kernel PANIC: KP: Asynchronous SError Interrupt

Kernel PANIC: KP: Oops: Fatal exception: __skb_ext_put

Kernel PANIC: KP: Oops: Fatal exception: dit_enqueue_reg_value_with_ext_lock

Kernel PANIC: KP: BRK handler: Fatal exception: dit_hal_ioctl

Kernel PANIC: KP: BRK handler: Fatal exception: dit_hal_get_netdev

Kernel PANIC: KP: BRK handler: Fatal exception in interrupt: comm:init, swapper/3-7

APC Watchdog: itom triggering err_fatal from HSIO USB31DRD_LINK to Refe

PMUCAL Watchdog: pmucal_local_disable: error on handling disable sequence. (pd: blkpwr_bo)

WARNING in drm_atomic_helper_commit_modeset_disables

WARNING in lwis_ioctl_handler

WARNING in gvoteable_cast_vote

WARNING in irq_set_irq_wake

WARNING in kbase_mem_pool_grow

WARNING in drm_mode_object_add

WARNING in gpio_to_desc

WARNING in corrupted

Emergency Restart

INFO: corrupted

RAMDUMP Mode

A ramdump is being collected.
This will take up to 5 minutes and you will be
prompted to upload it once your device boots into
Android.
Double tap <VOL_UP> and <VOL_DOWN> to skip
ramdump and continue booting.

For more information, see go/pixel-ramdump

Completed:71% | ETA:49 secs

reset message: KP: Oops: Fatal exception: comm:sys-executor
PC:dit_enqueue_reg_value_with_ext_lock+0x210/0x27c [exynos_dit]
LR:dit_enqueue_reg_value_with_ext_lock+0x200/0x27c [exynos_dit]
UID: 52a07ab6-e42c-ac43-9cc7-4a5201f778e
last kernel version: 5.10.81-g9dd1c348c400-ab8558925
aosp kernel version: 5.10.81-android12-9-ab8558925
build:
google/oriole_hwasan/oriole:12/SQ3A.220705.004.X1/0003385:userdebug/
dev-keys
RST_STAT: 0x80 - SYSTEM_SWRESET_SYSTEM
GSA_RESET_STATUS: 0x10 - GSA_INTERMEDIATE_RESET
Reboot reason: 0xbaba - Kernel PANIC
Reboot mode: 0x0 - Normal Boot

RAMDUMP Mode

A ramdump is being collected.
This will take up to 5 minutes and you will be
prompted to upload it once your device boots into
Android.
Double tap <VOL_UP> and <VOL_DOWN> to skip
ramdump and continue booting.

For more information, see go/pixel-ramdump

Completed:82% | ETA:44 secs

reset message: KP: BRK handler: Fatal exception: comm:sys-executor
PC:dit_hal_ioctl+0x93c/0x94c [exynos_dit] LR:dit_hal_ioctl+0x4a4/0x94c
[exynos_dit]
UID: 082270c4-6798-9240-a925-ba6cc289470c
last kernel version: 5.10.66-g2f8013a28c7d-ab8119957
aosp kernel version:
5.10.66-android12-9-00001-g51e133b6e4eb-ab8103786
build:
Android/aosp_oriole/oriole:Tiramisu/AOSP.MASTER/8310607:userdebug/te
st-keys
RST_STAT: 0x80 - SYSTEM_SWRESET_SYSTEM
GSA_RESET_STATUS: 0x10 - GSA_INTERMEDIATE_RESET
Reboot reason: 0xbaba - Kernel PANIC
Reboot mode: 0x0 - Normal Boot

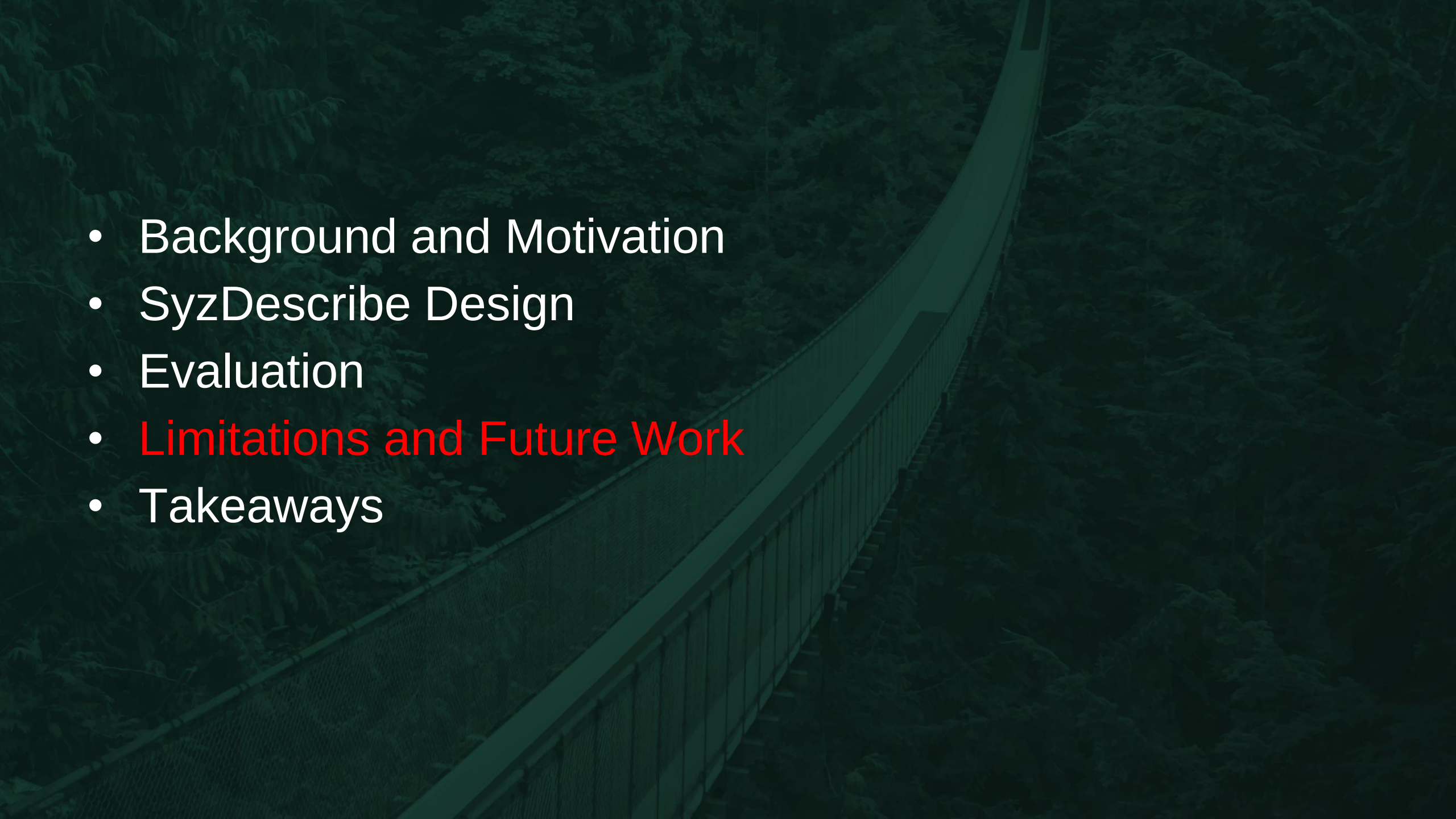
RAMDUMP Mode

A ramdump is being collected.
This will take up to 5 minutes and you will be
prompted to upload it once your device boots into
Android.
Double tap <VOL_UP> and <VOL_DOWN> to skip
ramdump and continue booting.

For more information, see go/pixel-ramdump

Completed:98% | ETA: 4 secs

reset message: itmon triggering err_fatal from HSIO USB31DRD_LINK to
Refe
UID: 413e1fd9-d5ae-b447-881a-2f90119b77a5
last kernel version: 5.10.66-g2f8013a28c7d-ab8119957
aosp kernel version:
5.10.66-android12-9-00001-g51e133b6e4eb-ab8103786
build:
Android/aosp_oriole/oriole:Tiramisu/AOSP.MASTER/8310607:userdebug/te
st-keys
RST_STAT: 0x1 - CLUSTER0_NONCPU_WDTRESET
GSA_RESET_STATUS: 0x10 - GSA_INTERMEDIATE_RESET
Reboot reason: 0xcba - APC Watchdog
Reboot mode: 0x0 - Normal Boot

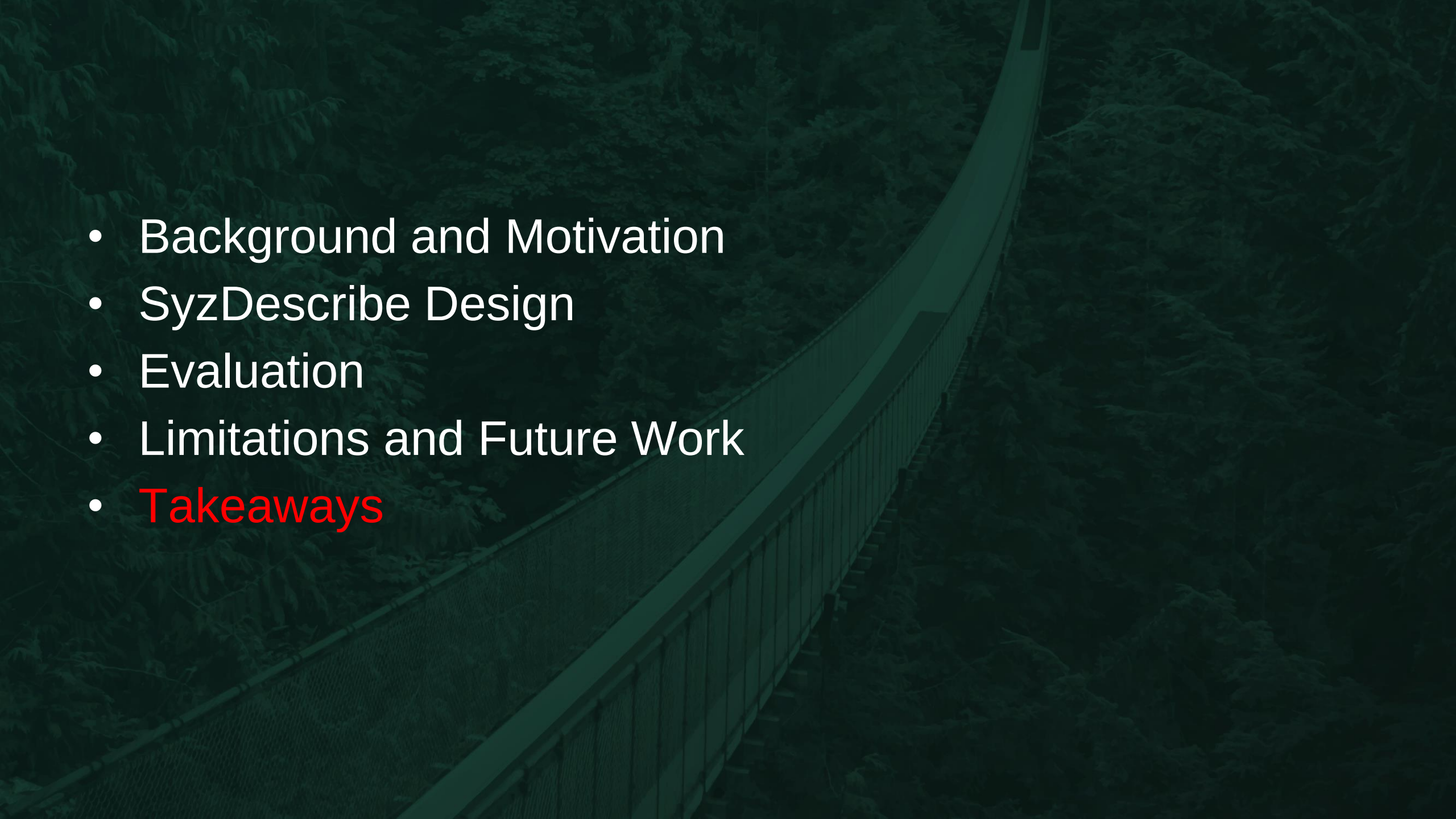
- 
- Background and Motivation
 - SyzDescribe Design
 - Evaluation
 - Limitations and Future Work
 - Takeaways

Limitations and Future Work

- Specific values or value ranges.
- Other syscalls: read(), write(), mmap()
- Other explicit dependencies.
- Merging syscall descriptions.

keep an eye on our open-source repo:

<https://github.com/seclab-ucr/SyzDescribe>

- 
- Background and Motivation
 - SyzDescribe Design
 - Evaluation
 - Limitations and Future Work
 - **Takeaways**

Takeaways

- We design a new tool, SyzDescribe, to generate syscall descriptions for Linux kernel drivers, which can significantly improve the effectiveness of fuzz testing for device drivers. This can help identify more bugs in the device drivers.
- We evaluate the accuracy of SyzDescribe against other solutions such as manually-curated syzkaller descriptions and showcase its effectiveness in the real world, based on the data gathered after analyzing hundreds of kernel drivers.
- Open source the implementation of SyzDescribe. This can help other researchers and practitioners in the field build upon our work.



LINUX
SECURITY
SUMMIT
NORTH AMERICA